Trabajo Práctico Tema 2: Diseño de la Estructura Operativa, Procesos y Marcos de trabajo

Asignatura: Administración de Sistemas de Información (ASI)
Nivel: 4º Año – Ingeniería en Sistemas de Información / Ingeniería Informática
Ciclo Lectivo: 2026
Modalidad: Grupal (Equipos de trabajo del Módulo 1)
Punto de Partida: Emprendimiento tecnológico formulado en el Módulo 1

🎯 1. Contexto del Desafío: El Escenario de Escalado

En el **Módulo 1**, cada equipo formuló las bases estratégicas de una startup de base tecnológica: se definió la propuesta de valor, se modeló el cliente ideal (*User Persona*), se analizó el entorno competitivo y se fijaron objetivos y métricas estratégicas (OKRs / KPIs).

🚀 La Situación de Negocio

Para este trabajo práctico, asumirán el siguiente escenario:

> **Su startup ha validado con éxito su Propuesta de Valor y su MVP (Producto Mínimo Viable) en el mercado real.**
> Los primeros usuarios activos han adoptado la solución, los indicadores iniciales son prometedores y la empresa ha cerrado una ronda de financiamiento para **escalar sus operaciones**.
>
> Como consecuencia, el negocio enfrenta una nueva realidad:
>
> 1. La base de clientes y el volumen transaccional crecen exponencialmente.
> 2. Se demandan nuevas funcionalidades complejas en paralelo a la necesidad de mantener el sistema actual con alta disponibilidad (99.9%).
> 3. El modelo de "3 o 4 fundadores haciendo de todo en una sola mesa" colapsó: aparecen cuellos de botella, fricción entre desarrollo y soporte, y riesgo de acumular deuda técnica descontrolada.

El reto de su equipo como Ingenieros en Sistemas:
Diseñar la **arquitectura operativa global** de la startup y las **metodologías y procesos ágiles** con los que operarán sus equipos de trabajo para garantizar entregas de valor continuas, predecibles y de alta calidad técnica.

🧠 2. Objetivos de Aprendizaje

Al completar este trabajo práctico, el equipo será capaz de:

1. Diseñar una **estructura organizacional y operativa escalable**, definiendo la topología de equipos y las cadenas de entrega de valor (*Value Streams*).
2. Seleccionar y justificar técnicamente las **metodologías de trabajo** adecuadas para la naturaleza específica de cada equipo.
3. Modelar **procesos operativos de punta a punta** (*End-to-End*), desde el descubrimiento de necesidades hasta el despliegue en producción.
4. Construir y configurar **radiadores visuales** (como tableros digitales en GitHub Projects, Jira, ClickUp, etc.) gobernados por políticas explícitas.
5. Institucionalizar **prácticas de ingeniería de software** y **marcos de métricas objetivas** para asegurar la sostenibilidad del sistema.

📋 4. Consignas Específicas de Desarrollo

El informe técnico consolidado deberá desarrollar de forma rigurosa y exhaustiva las siguientes **cuatro (4) partes**:

PARTE I: Organización a Alto Nivel y Flujo de Valor (Nivel Macro)

1.1. Diagnóstico de Escalado y Necesidades del Negocio

- Describan sintéticamente el estado actual de su startup (retomando el Módulo 1): qué problema resuelve y qué se validó en el MVP.
- Identifiquen los **3 principales desafíos operativos y técnicos** que enfrenta el sistema al multiplicar por diez su base de usuarios y su ritmo de entrega (e.g., concurrencia, seguridad, nuevos módulos funcionales, incidentes en producción, soporte al cliente).

1.2. Mapeo del Flujo de Entrega de Valor (*Value Stream Mapping* Conceptual)

- Modelen el flujo de punta a punta que recorre una idea o requerimiento, por ejemplo: Oportunidad de Negocio / Feedback -> Descubrimiento -> Desarrollo -> Verificación -> Despliegue -> Operación / Telemetría
- Indiquen claramente los puntos de contacto con el cliente ideal (*User Persona*) definido en el Módulo 1.

1.3. Topología y Estructura de Equipos de la Startup Escalada

- Definan la estructura organizativa de la ingeniería y producto de la empresa.
- Utilicen el concepto de **Topología de Equipos** (*Team Topologies*) o estructura de *Squads/Tribus*, especificando:

    - **Equipos orientados a flujo (*Stream-Aligned Teams*):** Equipos responsables de entregar valor directo en funcionalidades de producto (e.g., Squad de Pagos, Squad de Onboarding, Squad de Core/Algoritmo IA).
    - **Equipos de Plataforma (*Platform Team*) / DevOps / Arquitectura:** Equipos que brindan infraestructura, pipelines, seguridad y servicios base a los demás equipos.
    - **Equipo de Operaciones / Soporte / Mantenimiento:** Cómo se atiende el soporte técnico de incidentes no planificados.
- Presenten el **Organigrama Técnico Operativo** de la startup y la dotación proyectada de roles en cada equipo.

1.4. Mecanismos de Coordinación y Gobernanza Ágil Inter-Equipo

- Si hay múltiples equipos trabajando en el mismo producto o arquitectura:
    - ¿Cómo gestionan las dependencias técnicas entre equipos (APIs compartidas, base de datos común)?
    - ¿Qué mecanismo de sincronización utilizan? (e.g., *Scrum of Scrums*, sincronización semanal de entrega, mesas de arquitectura transversal).
    - ¿Cómo se visualiza el progreso? (Diagrama de Gannt, Tablero maestro o de portafolio, Niveles de vuelo)

PARTE II: Metodología y Mecánicas Operativas por Equipo (Nivel Micro / Detalle)

Cada equipo identificado en la Parte I puede tener naturalezas de trabajo distintas. En esta sección deben hacer zoom en el funcionamiento interno de los equipos de trabajo.

2.1. Selección Metodológica Diferenciada y Justificada

- Para cada equipo operativo definido (e.g., Equipo de Producto vs. Equipo de Plataforma/Operaciones), evalúen y seleccionen su marco de trabajo (**Scrum**, **Kanban**, **Scrumban****).
- **Justificación técnica obligatoria:** Expliquen por qué esa metodología es la óptima para ese equipo en particular y por qué fracasaría si intentaran aplicarle otra (e.g., *¿Por qué un equipo de operaciones e incidencias no debe operar con Sprints rígidos de Scrum? ¿Por qué un equipo de producto nuevo requiere la cadencia de entrega e inspección de Scrum?*).

2.2. Roles, Responsabilidades y Matriz RACI Ágil

- Para cada equipo, detallen los roles operativos. Por ejemplo:
    - **Product Owner (PO):** Alcance, priorización y relación con el negocio.
    - **Scrum Master / Agile Coach / Flow Facilitator:** Facilitación, remoción de bloqueos y optimización del flujo.
    - **Equipo de Ingeniería / Desarrolladores:** Tech Lead, Backend, Frontend, QA, DevOps/SecOps, etc.

- Confeccionen una **Matriz RACI Ágil** (*Responsible, Accountable, Consulted, Informed*) para las decisiones clave: definición de backlog, priorización, aprobación de PRs (*Pull Requests*), despliegue a producción y respuesta a incidentes críticos.

2.3. Sistema de Gestión Visual (Diseño del Tablero de Trabajo)

- Diseñen la topología de estados de su radiador visual de información.
- Definan las columnas de avance de tareas. Ejemplo: Backlog -> Refined (DoR) -> In Progress -> Code Review / PR -> QA / Testing -> Done (DoD)
- **Límites de Trabajo en Progreso (WIP Limits):**
    - Si lo consideran necesario, asignen límites numéricos explícitos de tareas simultáneas en las columnas intermedias (recordar la importancia de la Ley de Little).
- **Clases de Servicio (*Classes of Service*):**
    - Definan cómo circula el trabajo según su costo del retraso (*Cost of Delay*):
        - *Expedite:* Incidentes críticos o caídas de producción (evitan el WIP, atención en enjambre/*swarming*).
        - *Standard:* Historias planificadas del flujo normal.
        - *Fixed Date:* Entregas sujetas a plazos contractuales o regulatorios.
        - *Intangible:* Refactorización y tareas de deuda técnica.

2.4. Políticas Explícitas de Calidad (por ejemplo, DoR y DoD)

- **Definición de Preparado (*Definition of Ready - DoR*):** Lista de verificación explícita que una Historia de Usuario debe cumplir antes de que el equipo comience a programarla (e.g., cumple INVEST, criterios BDD redactados, mockups aprobados, dependencias técnicas resueltas).
- **Definición de Terminado (*Definition of Done - DoD*):** Lista de verificación técnica no negociable para dar por completada una historia (e.g., pruebas unitarias con cobertura $\ge 80%$, análisis estático de código sin vulnerabilidades críticas, revisión de código aprobada por un par, documentación de API actualizada, desplegado en ambiente de staging/producción).

2.5. Protocolo de Cadencias y Rituales del Equipo

- Confeccionen el cronograma de ceremonias/eventos del equipo principal. Por ejemplo, en Scrum:
    - **Planificación:** Duración, insumos y resultado esperado.
    - **Sincronización Diaria (*Daily*):** Dinámica de 15 minutos centrada en el tablero y en desbloquear el flujo.
    - **Revisión / Demostración (*Review*):** Presentación del incremento potencialmente desplegable ante stakeholders.
    - **Retrospectiva:** Análisis formal de procesos, personas y herramientas para generar mejoras accionables.

PARTE III: Backlog de Escalado y Prácticas Técnicas de Ingeniería

3.1. Arquitectura del Backlog de Escalado (Mínimo 8 Historias de Usuario)

- Definan al menos **dos (2) Épicas de Escalado** derivadas de los nuevos desafíos del negocio (e.g., *Épica 1: Arquitectura de autenticación escalable y gestión de roles empresariales*; *Épica 2: Automatización del motor de recomendaciones para alta concurrencia*).
- Redacten un catálogo de al menos **ocho (8) Historias de Usuario** representativas del sistema escalado.
- Cada historia debe cumplir rigurosamente con:
    1. **Estructura Estándar:** *"Como [User Persona], quiero [capacidad del sistema], para [beneficio concreto]."*
    2. **Checklist INVEST:** Justificación de que la historia es **I**ndependiente, **N**egociable, **V**aliosa, **E**stimable, **S**mall (pequeña/acotada) y **T**esteable.
    3. **Criterios de Aceptación en Formato BDD / Gherkin:**

        Escenario: [Nombre descriptivo]
        Dado que [contexto o precondición técnica]
        Cuando [el usuario realiza una acción específica]
        Entonces [el sistema responde con un resultado verificable]
    4. **Priorización**

3.2. Prácticas de Calidad Técnica e Ingeniería de Software (Ecosistema XP)

Seleccionen e instrumenten al menos **cuatro (4) prácticas obligatorias de Extreme Programming (XP)** adaptadas a su stack tecnológico. Por ejemplo:

- **Test-Driven Development (TDD):** Flujo Red-Green-Refactor y herramientas de pruebas unitarias.
- **Integración Continua / Despliegue Continuo (CI/CD):** Diseño del pipeline automatizado (e.g., GitHub Actions: linting -> build -> tests automáticos -> análisis SAST -> deploy).
- **Revisión de Código por Pares (*Peer Code Review*) o Programación en Parejas (*Pair Programming*):** Política de Pull Requests y reglas de aprobación.
- **Refactorización y Gestión de Deuda Técnica:** Política explícita de reserva de capacidad de ingeniería para refactorizar código sin romper funcionalidad existente.
- **Estándares de Codificación y Propiedad Colectiva del Código:** Guías de estilo y convenciones adoptadas por todo el equipo.

PARTE IV: Cuadro de Mando, Métricas de Flujo y Gobernanza Operativa

4.1. Métricas de Rendimiento de Flujo

Definan cómo medirá la startup la eficiencia de su proceso operativo utilizando indicadores cuantitativos objetivos, como:

- **Lead Time (Tiempo de Entrega):** Medición desde que surge la solicitud hasta que está disponible en producción.
- **Cycle Time (Tiempo de Ciclo):** Medición desde que el equipo inicia la tarea hasta que cumple la DoD.
- **Throughput (Tasa de Salida):** Número de ítems de trabajo completados por semana/Sprint.
- **Diagrama de Flujo Acumulado (*Cumulative Flow Diagram - CFD*):** Explicación de cómo utilizarán el CFD para monitorear la estabilidad del WIP e identificar tempranamente cuellos de botella.

4.2. Métricas de Estabilidad Técnica y Despliegue (Métricas DORA)

Especifiquen metas cuantitativas para los cuatro indicadores clave de rendimiento de entrega de software (*DevOps Research and Assessment*):

1. **Frecuencia de Despliegue (*Deployment Frequency*):** ¿Con qué frecuencia se libera código a producción?
2. **Tiempo de Cambio (*Lead Time for Changes*):** ¿Cuánto tarda un commit en llegar a producción de forma segura?
3. **Tasa de Fallos en Cambios (*Change Failure Rate*):** ¿Qué porcentaje de despliegues genera incidentes que requieren rollback o hotfix?
4. **Tiempo de Recuperación (*Mean Time to Restore - MTTR*):** ¿Cuánto tarda el equipo en restaurar el servicio ante una caída?

4.3. Métricas de Negocio y de Impacto (*Outcomes*)

Para asegurar que la eficiencia operativa se traduzca en valor real (evitando construir rápido cosas que nadie usa), definan cómo conectarán el trabajo de desarrollo con los objetivos de negocio del Módulo 1, como:

- **Adopción y Uso del Producto:** Métricas de tracción como Usuarios Activos Diarios/Mensuales (*DAU / MAU*), tasa de retención o tasa de activación (*Time-to-Value*).
- **Impacto Económico o de Conversión:** Indicadores directos del modelo de negocio (ej. tasa de conversión en el funnel, *Monthly Recurring Revenue - MRR*, tasa de abandono / *Churn Rate*).
- **Satisfacción y Calidad Percibida:** Métricas de experiencia del cliente (*Customer Journey*) como *NPS (Net Promoter Score)* o *CSAT (Customer Satisfaction Score)*.
- **Alineación con OKRs:** Explicar cómo el avance del Backlog impacta directamente en al menos un Objetivo y Resultado Clave (*Key Result*) definido en el Módulo 1.

📦 5. Entregables y Criterios de Presentación

Cada grupo deberá entregar:

1. **Informe Técnico Consolidado (PDF):**
    - Documento formal que desarrolle con profundidad todas las secciones de las Partes I, II, III y IV.

- Incluir esquemas gráficos cuando corresponda: organigrama, flujo de valor, topología de equipos, diseño conceptual del pipeline de CI/CD, etc.

2. **Enlace a radiadores de información:**
    - URL de acceso público (o invitación a la cátedra) a un tablero creado en una herramienta real de la industria (como **GitHub Projects**, **Jira Software**, **ClickUp**, **Trello**, etc.). De ser necesario, debe utilizarse un tablero para cada equipo.
    - Si aplica, enlace al tablero maestro, diagrama de Gantt o el recurso de alto nivel utilizado para gestionar y visualizar las iniciativas en forma global.
    - Cada tablero debe evidenciar:
        - Columnas configuradas.
        - Carga de las **8 Historias de Usuario mínimas**, con sus criterios de aceptación BDD documentados dentro de cada tarjeta.
        - Políticas publicadas en el tablero o en su documentación accesible (como **DoR** y **DoD**).
        - Al menos una tarjeta etiquetada como *Expedite* o *Bug Crítico* mostrando cómo opera la clase de servicio.